Git & GitHub

Version control for reproducible science

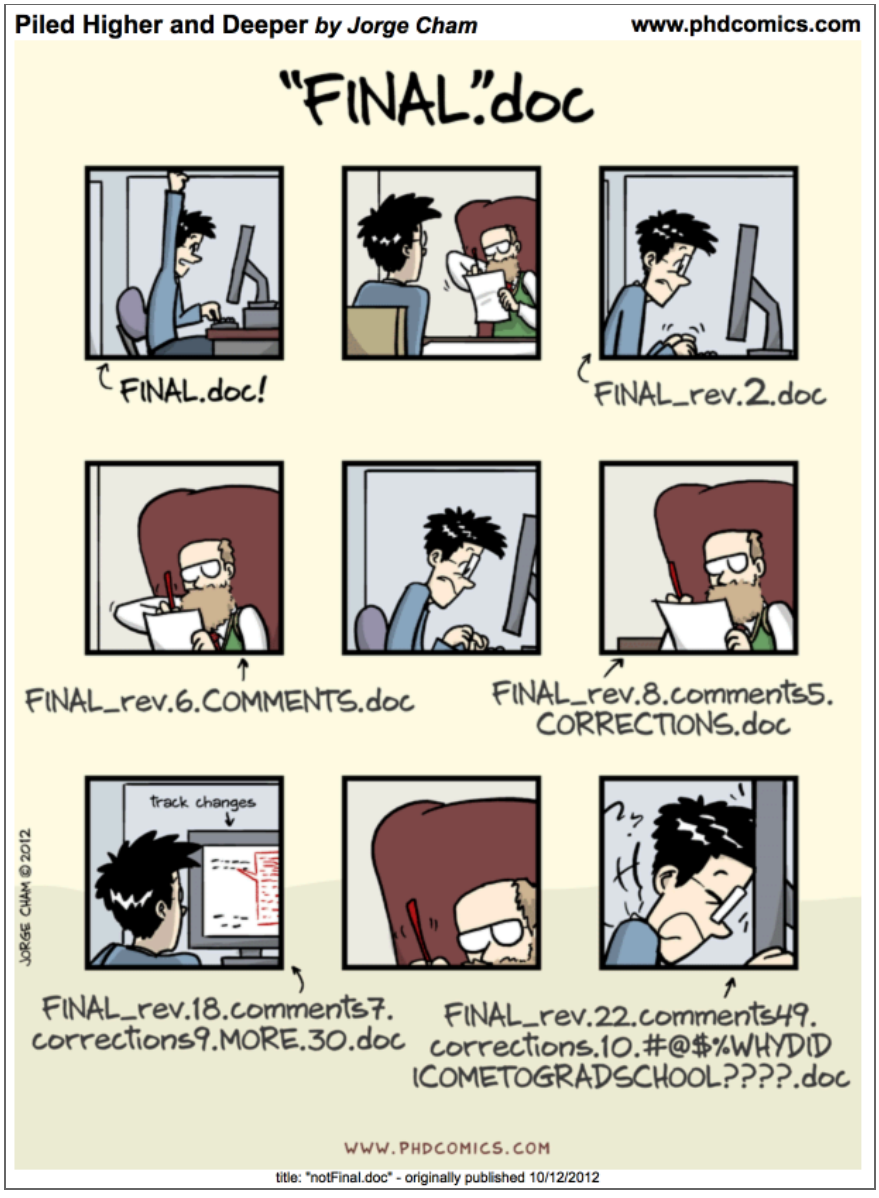
NCEAS Learning Hub

Learning Objectives

By the end of this session, you will be able to:

- Apply the principles of Git to **track and manage changes** in a project
- Understand how **local working files, Git, and GitHub** work together
- Use the Git workflow: **pull** → **stage** → **commit** → **push**
- Create and configure a **Git repository** on GitHub
- **Clone** a repository and work with Git locally in RStudio

The problem with filenames



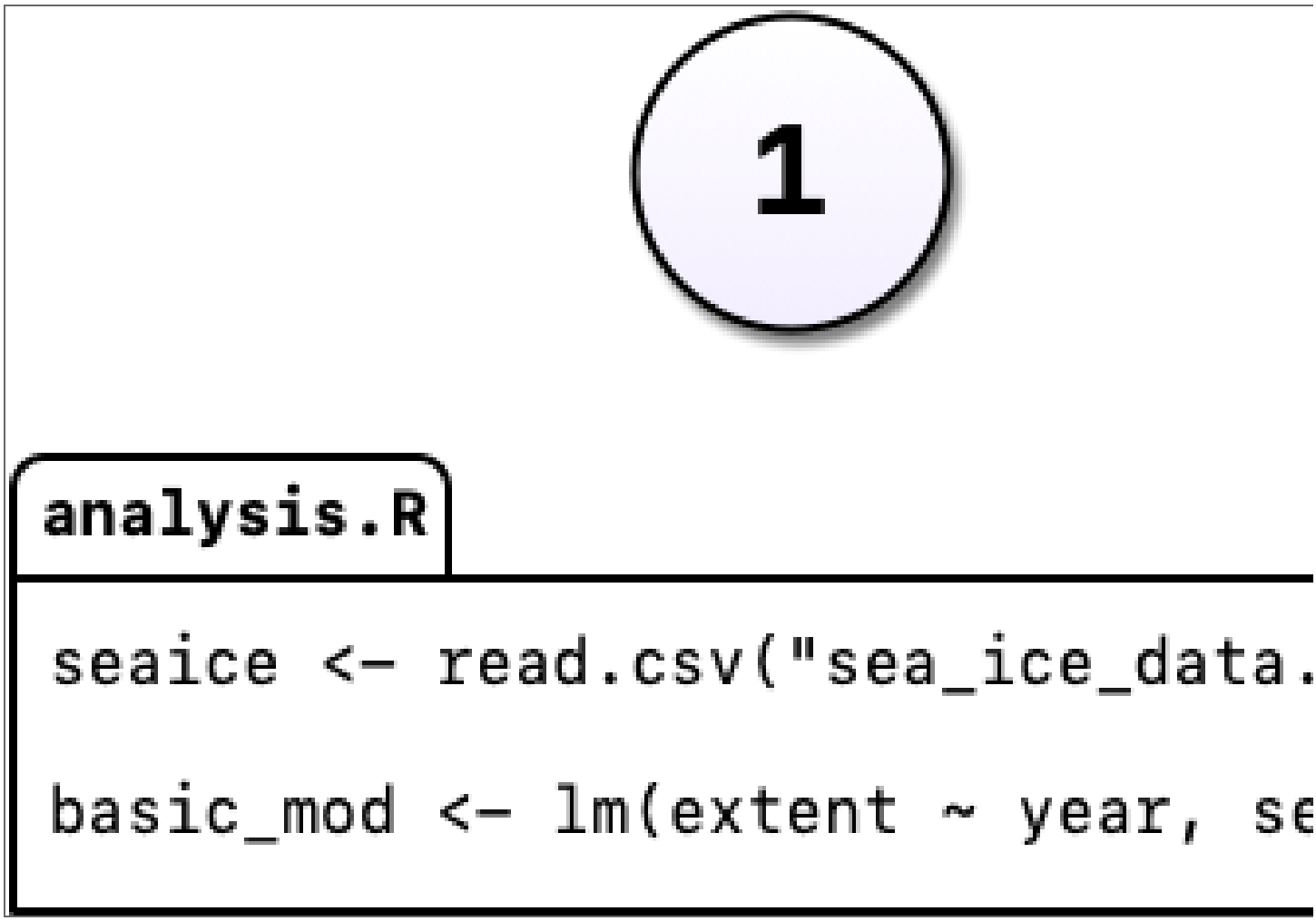
Why version control?

Every file in the scientific process **changes**. Manuscripts are edited. Figures get revised. Code gets fixed when bugs are discovered — sometimes those fixes lead to more bugs.
Version control provides an **organized and transparent** way to track changes in code and files.
Benefits:

- **Maintain a history** of project development while keeping your workspace clean
- **Facilitate collaboration** and transparency when working on teams
- **Explore bugs or new features** without disrupting your team's work

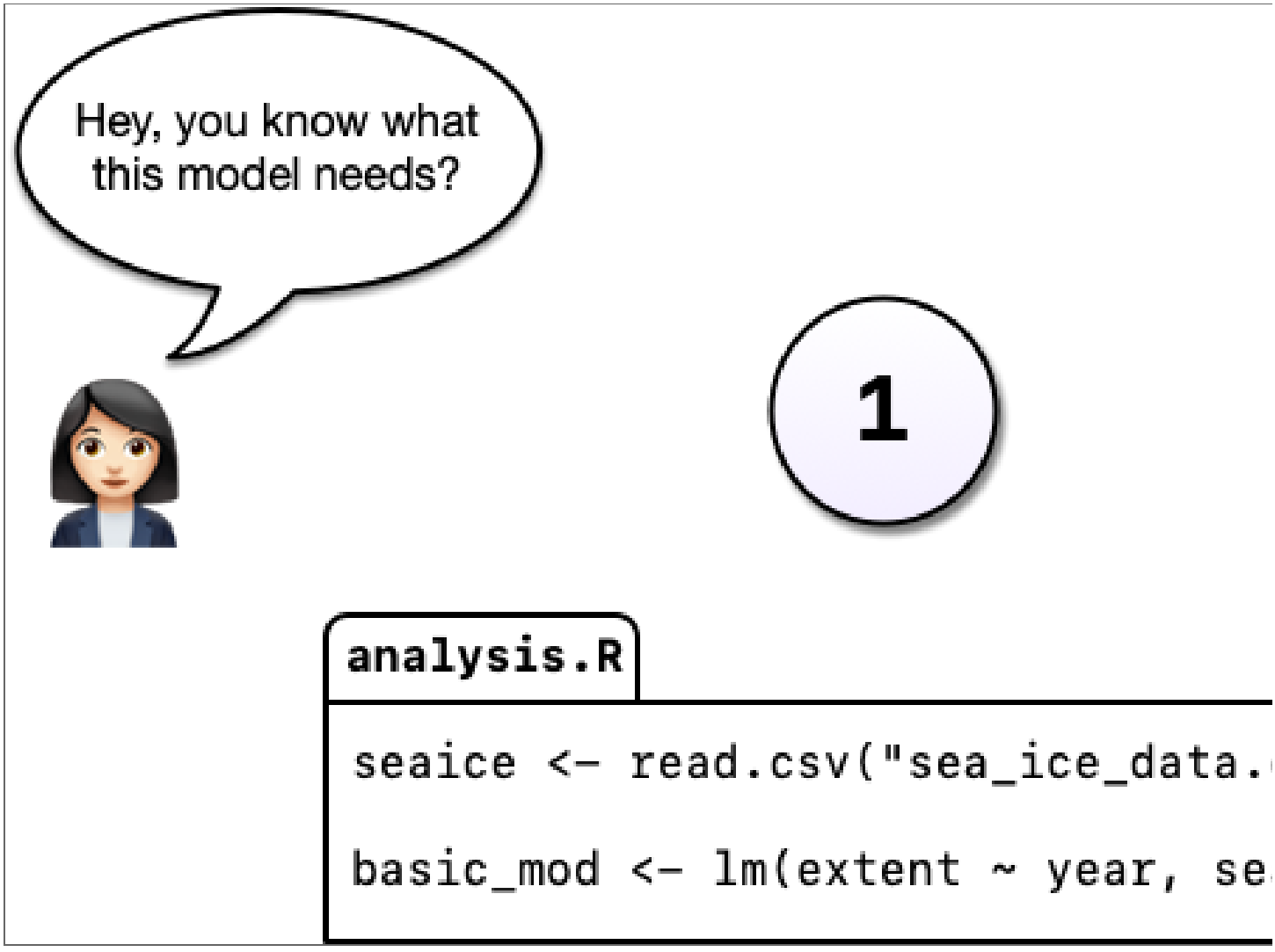
A motivating example

You're working on an analysis in R and you've got it into a state you're pretty happy with. We'll call this **version 1**.



A motivating example

The next day, you have an email from your boss: *"Hey, you know what this model needs?"*



A motivating example

You add it to the model — but you're worried about losing the old version. So you **comment out** the old code and add a warning.

Commenting out code you don't want to lose is common — but it's really hard to understand *why* you did it when you come back later.

A better way: commit the change

Instead of commenting out old code, we can **change the code in place** and tell Git to commit our change. Now we have two distinct versions — and we can always see what the previous version looked like.

Git also tracks **who**, **when**, and **where** the change was made.

Experimenting without fear

After committing a third version, a colleague has an idea: *“What if you used machine learning instead?”*

Without Git, you’d copy `analysis.R` to `analysis-ml.R` — and now shared code has to be updated in two places.

Enter: branches

With Git, you can start a **branch**. Branches let you confidently experiment — all while leaving the old code intact and recoverable.

Working in parallel

With Git and branches, we can continue developing our **main analysis** at the same time as experimental branches.

What Git gives you

- **Eliminate** cryptic filenames and comments to track your work
- Provide **detailed descriptions** of changes through commit messages
- Work on multiple **branches** simultaneously and optionally merge them
- **Access and even execute older versions** of your code
- Have **multiple people working concurrently**, with the ability to merge everyone's changes together

What exactly are Git and GitHub?

Git — version control software

- Open-source, **runs locally** on your computer
- Tracks file changes and project history
- Works via command line or RStudio's Git tab
- Does not require a network connection

GitHub — online platform for Git

- **Cloud hosting** for Git repositories

- Web interface, issue tracking, collaboration tools
- Pull requests, code review, project management
- Social features: follow, star, discover code

Git vs. GitHub

Git is the engine.
GitHub is the garage where you park, share, and collaborate on your car.

- Most Git operations happen **locally** — you don’t need the internet
- GitHub is where you **back up**, **share**, and **collaborate**
- You can use Git without GitHub; GitHub requires Git

Normal folder vs. Git repository

Let's say you create a folder `myFolder/` with two files: `myData.csv` and `myAnalysis.R`.
Right now, those files are **not version-controlled**. If you break something, there's no going back.
When you **initialize** a Git repository, a hidden `.git/` folder is created inside. That `.git/` folder *is* the repository — it stores every snapshot you've ever committed.

> **Warning**
>
> If you delete `.git/`, you delete your entire project history.

The Git repository

The Git workflow

Locally — creating snapshots:

1. **Make changes** to a file in your working directory
2. **Stage** the file: `git add myAnalysis.R` (or `git add .` for all)
3. **Commit** the file: `git commit -m "describe your change"`

Syncing with GitHub:

4. **Push** local commits to GitHub: `git push`
5. **Pull** changes from GitHub: `git pull`

> **Tip**
>
> Always **pull before you push** — especially when collaborating!

Visualizing the workflow

Artwork by Allison Horst

What a GitHub repository looks like

File listing, last modified dates, who made the most recent changes — all visible at a glance.

Inspecting the commit history

Drill into **commits** and see the full history of every change ever made to the project.

Seeing exactly what changed

Drill into any single commit to see **line-by-line** changes: additions in green, deletions in red.

This is what `FINAL_v3_please_please.R` can never give you.

Git vocabulary: the essentials

Term	Command	What it does
Stage	`git add [file]`	Mark file(s) for next commit
Commit	`git commit -m "msg"`	Record a snapshot with a message
Pull	`git pull`	Fetch + merge changes from remote
Push	`git push`	Send local commits to GitHub
Status	`git status`	See what's staged/unstaged
Clone	`git clone [url]`	Copy a remote repo locally

Bookmark the full vocabulary tables in the chapter — including advanced commands like `branch`, `merge`, `checkout`, and `log`.

Writing good commit messages

- **Be descriptive and concise** — convey the purpose in a few words
- **Use imperative tense** — “Add analysis” not “Added analysis”
- **Keep subject line under 50 characters**
- **Focus on the “what” and “why”**, not just what you clicked

> **Examples**
>
> ❌ `Updates`
> ❌ `Fixed stuff`
> ✅ `Fix mean calculation to exclude NA values`
> ✅ `Add soil pH summary to exploratory analysis`

Exercise 1: Create a remote repository

Setup

1. Log in to **GitHub**
2. Click **New repository**
3. Name it `{FIRSTNAME}_test`
4. Add a short description
5. Check the box to add a `README.md`
6. Add a `.gitignore` file using the **R** template
7. Set the License to **Apache 2.0**

Then: navigate to `README.md`, click the **pencil icon**, add a level-2 header called “Purpose” with a few bullet points, and commit the change.

Your first repository

Editing directly on GitHub

Your first versioned commit

The landing page shows all files, when each was last edited, and the **commit message** for each file's most recent change — this is why descriptive commit messages matter!

Exercise 2: Clone your repository

The copy on GitHub is the **origin repository**. Now we bring it down to your local machine.

> **Setup**
>
> 1. Click the green **Code** button on your GitHub repo
> 2. Copy the URL
> 3. In RStudio: **File** → **New Project** → **Version Control**
> 4. Paste the URL → press **Tab** to auto-fill the directory name
> 5. Click **Create Project**

RStudio with Git

You'll see a **Git tab** in RStudio — this is your version control panel.
`??` means a file is untracked (Git doesn't know about it yet).

Practice the full workflow

Make a change to your `README.md` in RStudio, then:

1. **Stage** — check the box next to the file
2. **Commit** — write a descriptive message
3. **Pull** — always before pushing!
4. **Push** — send to GitHub

Inspecting history in RStudio

Click **History** in the Git tab to see every commit — identical to what you'd see on GitHub.

One configuration step

To suppress a Git warning about pull strategy, run this once in the **Terminal** pane (not the Console):

```
git config pull.rebase false
```

This tells Git to use the default merge strategy when pulling. You'll only need to do this once per repository.

What you can now do

- Explain the difference between **Git** (local) and **GitHub** (remote)
- Understand how the `.git/` **folder** stores your project history
- Use the **Stage** → **Commit** → **Pull** → **Push** workflow
- Create a repository on GitHub and **clone** it to RStudio
- Write **descriptive commit messages** that make your history meaningful

Next week: Exercise 3 — setting up Git on an existing project

Go further with Git

- **Happy Git and GitHub for the useR** — Jenny Bryan; the gold standard for R users
- **git documentation** — comprehensive command reference
- **Git vocabulary tables** — Essential and Advanced command tables in the course chapter
- **GitHub Docs** — **docs.github.com**

The chapter's "Go further with Git" section has additional exercises and resources.

Speaker notes